

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

ОТЧЁТ ПО ЗАДАНИЮ №6
«Сборка многомодульных программ. Вычисление корней
уравнений и определенных интегралов»

Вариант: 4 / 1 / 3

Выполнил:
студент 102 группы
Лукашев Д. А.

Преподаватели:
Калинин Г. М.
Цыбров Е. Г.

Москва
2026

Содержание

Постановка задачи	2
Математическое обоснование	3
Результаты экспериментов	4
Структура программы и спецификация функций	5
Сборка программы (Make-файл)	6
Отладка программы, тестирование функций	8
Программа на Си и на Ассемблере	9
Анализ допущенных ошибок	10
Список цитируемой литературы	11

Постановка задачи

В данной работе решалась задача вычисления площади плоской фигуры, ограниченной тремя заданными кривыми.

Требуется реализовать численный метод, позволяющий вычислять площадь плоской фигуры. В качестве метода численного интегрирования используется формула Симпсона (парабол). Вершины фигуры (точки пересечения кривых) необходимо искать численно, используя метод деления отрезка пополам (дихотомии).

Отрезки для применения метода нахождения корней должны быть вычислены аналитически на основе графического и математического анализа заданных функций. Итоговая точность вычисления площади должна составлять $\varepsilon = 0.001$.

Математическое обоснование

Согласно варианту 4, фигура ограничена следующими функциями:

1. $f_1(x) = e^x + 2$
2. $f_2(x) = -\frac{1}{x}$
3. $f_3(x) = -\frac{2(x+1)}{3}$

Графики данных функций представлены на рис. 1.

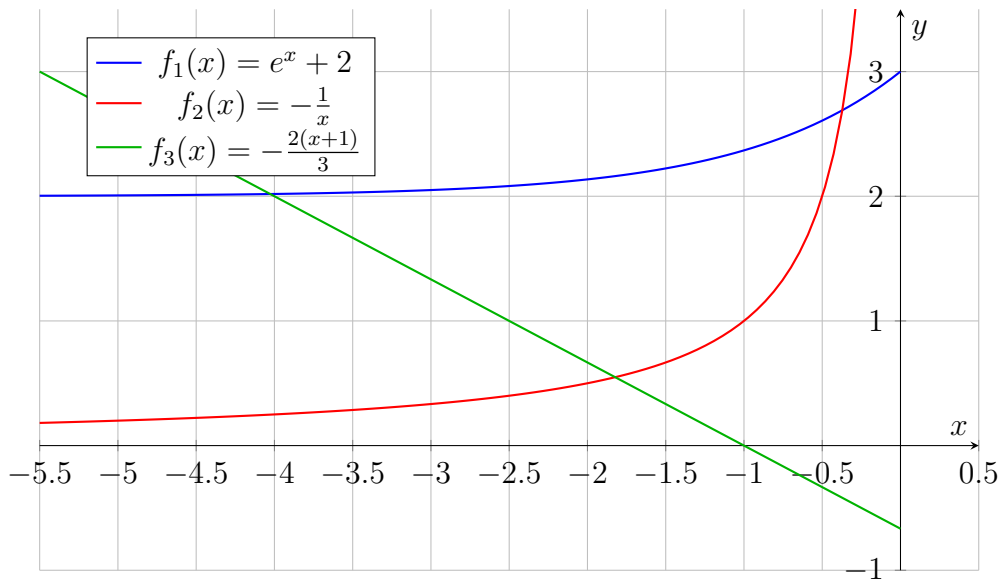


Рис. 1: Плоская фигура, ограниченная графиками заданных уравнений

Отрезки для применения метода дихотомии были определены аналитически путем поиска участков, на концах которых функция разности меняет знак:

- Пересечение f_1 и f_3 : интервал $[-5.0, -3.0]$
- Пересечение f_3 и f_2 : интервал $[-2.5, -0.5]$
- Пересечение f_1 и f_2 : интервал $[-1.0, -0.1]$

Обоснование выбора ε_1 и ε_2 : Согласно условию, площадь должна быть вычислена с точностью $\varepsilon = 0.001$. Общая погрешность ΔS складывается из погрешности квадратурных формул ΔS_{int} и погрешности за счет неточного вычисления границ интегрирования ΔS_{roots} .

Известно [1], что изменение площади за счёт смещения границы интегрирования имеет второй порядок малости от погрешности корня. Поэтому при выборе $\varepsilon_1 = 0.001$ погрешностью ΔS_{roots} можно пренебречь.

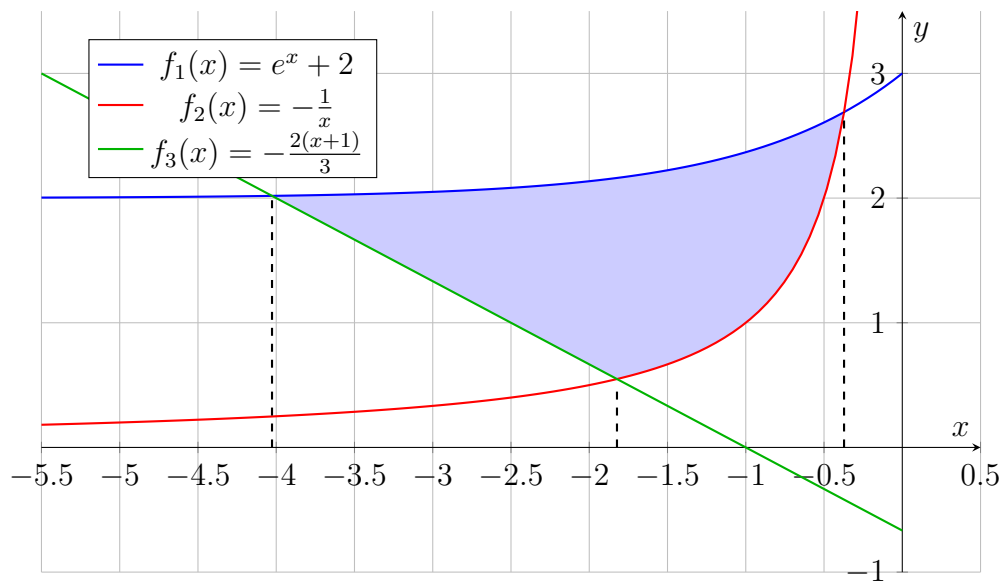
Площадь вычисляется как комбинация трёх интегралов. Максимальная погрешность суммы достигает $3\varepsilon_2$. Чтобы удовлетворить требованию задачи, необходимо $\Delta S_{int} \leq 3\varepsilon_2 \leq 0.001$. Следовательно, достаточной точностью для интегрирования является $\varepsilon_2 = 0.0001$.

Результаты экспериментов

Координаты найденных точек пересечения кривых (с точностью $\varepsilon_1 = 0.001$) представлены в таблице 1.

Кривые	Координата x
f_1 и f_3	-4.026855
f_3 и f_2	-1.822876
f_1 и f_2	-0.372021

Таблица 1: Абсциссы точек пересечения кривых



Итоговая площадь полученной фигуры, вычисленная по формуле Симпсона как сумма и разность определенных интегралов, составила: **$S \approx 3.563621$** .

Структура программы и спецификация функций

Программа разделена на три основных компонента:

- `main.c` — главный модуль. Обрабатывает аргументы командной строки, координирует вызовы численных методов, вычисляет общую площадь.
- `methods.c` — реализация численных методов. Содержит функцию `root` (метод дихотомии) и `integral` (формула Симпсона).
- `functions.asm` — модуль на языке Ассемблера с использованием сопроцессора x87, вычисляющий значения функций f_1, f_2, f_3 .

Разбиение программы и связи между компонентами проиллюстрированы графически на рис. 2.

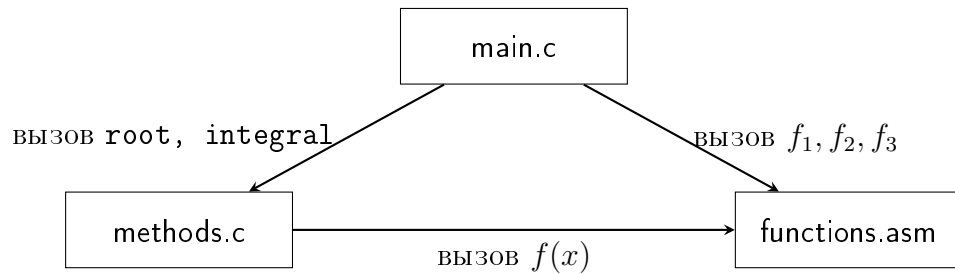


Рис. 2: Архитектура многомодульной программы

Сборка программы (Make-файл)

Сборка программы осуществляется утилитой `make`. Зависимости между модулями показаны на рис. 3:

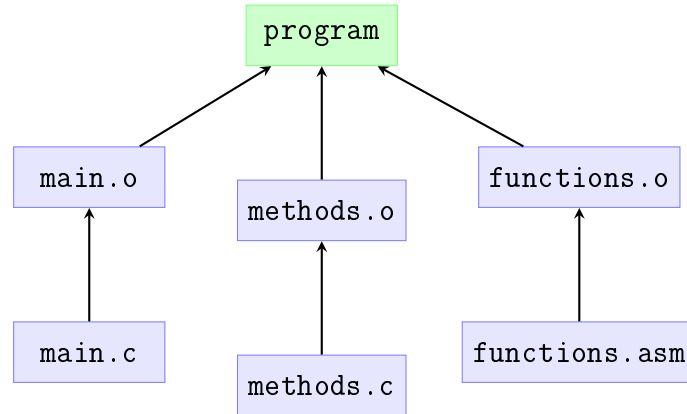


Рис. 3: Диаграмма зависимостей файлов при сборке

Текст Makefile приведен ниже:

```
1 .PHONY: all clean prepare
2
3 CC = gcc
4 CFLAGS = -m32 -O2 -Wall
5 ASM = nasm
6 ASMFLAGS = -f elf32
7
8 all: program
9
10 program: main.o methods.o functions.o
11     $(CC) $(CFLAGS) -o program main.o methods.o functions.o
12
13 main.o: main.c methods.h functions.h
14     $(CC) $(CFLAGS) -c main.c
15
16 methods.o: methods.c methods.h
17     $(CC) $(CFLAGS) -c methods.c
18
19 functions.o: functions.asm
20     $(ASM) $(ASMFLAGS) -o functions.o functions.asm
21
22 clean:
23     rm -f *.o program
```

В Makefile предусмотрена возможность задания компилятора для исходных файлов на языке C, а также ассемблерного компилятора вместе с соответствующими флагами сборки.

Для повышения удобства работы с проектом также была добавлена цель, обеспечивающая компиляцию итогового PDF-документа из исходного файла

.tex с использованием pdflatex.

Кроме того, с целью упрощения подготовки рабочего окружения была реализована цель rgerage, выполняющая обновление системных ссылок на пакеты и установку инструментов, необходимых для сборки и использования проекта.

Отладка программы, тестирование функций

Тестирование численных методов проводилось в режиме модульного тестирования на функциях f_1, f_2, f_3 , заданных в варианте.

Тестирование метода поиска корней (деление пополам):

1. **Тривиальный тест (линейная функция):** Поиск корня уравнения $f_3(x) = 0$ на отрезке $[-2, 0]$. Аналитически: $-\frac{2(x+1)}{3} = 0 \Rightarrow x = -1.000$. Результат программы ($\varepsilon_1 = 0.001$): -1.000 .
2. **Тест нелинейного уравнения:** Поиск пересечения $f_2(x) = f_3(x)$ на отрезке $[-2.5, -1.0]$. Аналитически: $-\frac{1}{x} = -\frac{2(x+1)}{3} \Rightarrow 2x^2 + 2x - 3 = 0$. Корень на заданном отрезке: $x = \frac{-2 - \sqrt{4 - 4 \cdot 2 \cdot (-3)}}{4} = \frac{-2 - \sqrt{28}}{4} \approx -1.823$. Результат программы ($\varepsilon_1 = 0.001$): -1.823 .
3. **Тест трансцендентной функции:** Поиск корня уравнения $f_1(x) = 3$ на отрезке $[-1, 1]$. Аналитически: $e^x + 2 = 3 \Rightarrow e^x = 1 \Rightarrow x = 0.000$. Результат программы ($\varepsilon_1 = 0.001$): 0.000 .

Тестирование метода интегрирования (метод Симпсона):

1. **Тривиальный тест (линейная функция):** Вычисление интеграла от $f_3(x)$ на отрезке $[-2, -1]$. Аналитическое вычисление по формуле Ньютона-Лейбница:

$$\begin{aligned} \int_{-2}^{-1} -\frac{2}{3}(x+1) dx &= -\frac{2}{3} \left[\frac{x^2}{2} + x \right]_{-2}^{-1} = \\ &= -\frac{2}{3} \left(\left(\frac{1}{2} - 1 \right) - \left(\frac{4}{2} - 2 \right) \right) = -\frac{2}{3} \left(-\frac{1}{2} - 0 \right) = \frac{1}{3} \approx 0.333 \end{aligned}$$

Результат программы ($\varepsilon_2 = 0.0001$): 0.333 .

2. **Интегрирование гиперболы:** Вычисление интеграла от $f_2(x)$ на отрезке $[-2, -1]$. Аналитическое вычисление:

$$\begin{aligned} \int_{-2}^{-1} -\frac{1}{x} dx &= -\left[\ln|x| \right]_{-2}^{-1} = -(\ln(1) - \ln(2)) = \\ &= -(0 - 0.6931) \approx 0.693 \end{aligned}$$

Результат программы ($\varepsilon_2 = 0.0001$): 0.693 .

3. **Интегрирование экспоненты:** Вычисление интеграла от $f_1(x)$ на отрезке $[0, 1]$. Аналитическое вычисление:

$$\begin{aligned} \int_0^1 (e^x + 2) dx &= \left[e^x + 2x \right]_0^1 = (e^1 + 2) - (e^0 + 0) = \\ &= (e + 2) - (1 + 0) = e + 1 \approx 2.718 + 1 = 3.718 \end{aligned}$$

Результат программы ($\varepsilon_2 = 0.0001$): 3.718 .

Все тесты пройдены успешно, результаты работы численных методов полностью совпали с аналитическими расчетами.

Программа на Си и на Ассемблере

Исходные тексты программы (модули на языке С, исходные коды ассемблерных функций, заголовочные файлы и сборочный скрипт Makefile) имеются в архиве, который приложен к этому отчету. В качестве меры предосторожности от блокировки вложения почтовыми службами используется зашифрованный архив с паролем «123».

Анализ допущенных ошибок

В процессе выполнения работы возникли и были решены следующие технические и математические проблемы:

- Отсутствие прямой инструкции вычисления e^x в x87. Проблема решена через использование представления $2^{x \log_2 e}$ с разбиением на целую и дробную части и применением команд `f2xm1` и `fscale`.
- Сбои при вызове ассемблерных функций из-за утечек стека x87. Проблема решена тщательным контролем и очисткой стека сопроцессора перед инструкцией `ret`.
- Неверный порядок операндов при делении $1/x$. Исправлено аккуратным использованием `fdiv` с нужным направлением операции.
- Выбор порядка пределов интегрирования. При интегрировании «справа налево» площадь получалась отрицательной. Исправлено корректной сортировкой точек пересечения.

Список цитируемой литературы

Список литературы

- [1] Ильин В.А., Садовничий В.А., Сендов Бл.Х. *Математический анализ. Часть 1. Книга 2. Глава 9 (9.4 - 9.6)* - Москва: Наука, 1985.